Certified Contractions and Depth Bounds for Full Optimal Binary Search Trees

Research checkpoint: exact preprocessing and unsuccessful relaxations

OpenAI Codex Sydney

Problem selection and sponsorship: the repository owner

October 18, 2026

Abstract

We investigate exact construction of optimal binary search trees with weights on both keys and gaps. We give sufficient conditions for contracting a single key, or a contiguous block of keys and gaps, without changing the optimum except for an explicitly known additive cost. A linear preprocessing pass selects certified blocks from a Cartesian-tree hierarchy and chooses a size threshold minimizing the number of subsequent dynamic-programming states. If the surviving core has m keys and the selected blocks have sizes k_t, construction takes $O(n + m^2 + \sum_t k_t^2)$ arithmetic operations. The method remains quadratic in the worst case. On one seeded positive-weight instance with 512 keys and sparse heavy separators, it reduced the number of DP states from 131,328 to 10,344; ordinary random mixed weights benefited little. We also document an order-free Huffman relaxation, its failed sparse-DP implementation, and exact counterexamples to proposed local certificates. A second relaxation prices depth-count constraints along the ordered key sequence. An affine-tail lemma evaluates it in $O(n \log n)$ time without assuming the optimal tree is shallow, but a five-key counterexample proves that this relaxation is not universally exact. These are research results and reproducible observations, not a resolution of general subquadratic construction. Novelty of the contraction conditions has not been established.

1 Model, scope, and prior work

Keys have fixed inorder positions $0, \dots, n-1$ and nonnegative weights $p_0, \dots, p_{n-1}$. The $n+1$ gaps have weights $q_0, \dots, q_n$. A key at depth d contributes $p(d+1)$; a gap at depth d contributes qd. Charging an extra comparison at each gap adds the tree-independent constant $\sum q_i$.

For a half-open key interval $[i, j)$, write

$$W(i,j) = \sum_{r=i}^{j-1} p_r + \sum_{r=i}^{j} q_r, \qquad C(i,i) = 0.$$

The exact recurrence is

$$C(i,j) = W(i,j) + \min_{i \le r < j} \{C(i,r) + C(r+1,j)\}.$$

Knuth's monotone-root optimization computes the table in $O(n^2)$ time [1]. The central obstacle here is the number of states, not merely the cost per state. We retain both kinds of weights throughout.

Larmore gives subquadratic exact construction under restrictions on long runs of very low frequency items, together with an approximation tradeoff [2]. Karpinski, Larmore, and Rytter combine exact small intervals with a bounded top-level construction [3]. These results are essential context: an improvement on a restricted family must not be described as the first subquadratic result. This checkpoint does not make a comprehensive priority claim. The conditions below were derived in this investigation; a targeted literature search did not establish whether equivalent rules already exist.

All running-time bounds below count arithmetic operations and comparisons. They are not bit-complexity bounds for unbounded integer weights.

2 A sharper single-key contraction

A key is a *cherry* when its two children are external nodes. Replacing a cherry with one gap is exact: if the key and its gaps have weights p, x, y, their total is $s = p + x + y$. A cherry rooted at depth d contributes $s(d+1)$, whereas the replacement gap contributes sd.

Theorem 1 (Single-key rule)**.** *Let a key of weight p have adjacent gap weights x, y. If its immediate predecessor and successor keys have weights a, b and*

$$\min(a, b) \;\geq\; 2p + x + y,$$

there is an optimal tree in which this key is a cherry. At the left endpoint, it suffices that the successor weight is at least $p + x$; at the right endpoint, it suffices that the predecessor weight is at least $p + y$. Consequently contraction to a gap of weight $s = p + x + y$ satisfies $\mathrm{OPT}(\textit{original}) = \mathrm{OPT}(\textit{contracted}) + s$.

Proof. Consider the key at depth d in any tree. If its left subtree is nonempty, the immediate predecessor has depth $d + l$, $l \geq 1$. If its right subtree is nonempty, the immediate successor has depth $d + r$, $r \geq 1$. Set the corresponding distance to zero for an empty internal side.

When $l \geq 1$, move the predecessor to the chosen key's position, replacing the predecessor's old position by its left child. Insert the chosen key as a cherry at the leftmost gap of the former right subtree. Inorder order is preserved. All other outcomes either retain their depths or become shallower. The predecessor saves la, and the cost change is at most

$$-la + (r+1)p + (r-l+1)x + y. \tag{1}$$

The mirrored successor exchange has change at most

$$-rb + (l+1)p + x + (l-r+1)y. \tag{2}$$

If both sides are nonempty and $l \geq r$, substituting $a \geq 2p + x + y$ in (1) gives

$$(r + 1 - 2l)(p + x) + (1 - l)y \leq 0.$$

Use (2) when $r \geq l$. If only the left side is nonempty, $r = 0$ and $a \geq p + y$ makes (1) nonpositive. The other case is symmetric. The stated interior condition implies these one-sided conditions. At a global endpoint only the appropriate one-sided case is possible. With both sides empty the key is already a cherry.

Applying the exchange to an optimum proves the existence claim. Contraction subtracts exactly s. Conversely, every contracted tree expands to a valid original tree at additional cost s, proving equality. □

The weaker condition $\min(a, b) \geq p + x + y$ is false. For $p = (24, 14, 24)$ and $q = (1, 0, 9, 0)$, the middle key has local mass 23 and neighbors of weight 24. Contracting it forces cost 142, while the unrestricted optimum costs 130.

Maintain a doubly linked list of remaining keys and weighted gaps. Initially queue every key; after a contraction, only its predecessor and successor need rechecking. At most $3n$ queue entries are processed. Saving the contracted tree at the new gap permits reconstruction. Thus this pass takes $O(n)$ operations and space, and a Knuth solve on the remaining m keys gives total time $O(n + m^2)$. No confluence or minimum-core-size property is claimed.

3 Whole-block contraction

Let $B = [i, j)$ be a nonempty proper interval, including its endpoint gaps q_i, q_j. Put $W = W(i, j)$ and $C = C(i, j)$, with costs measured from a stand-alone root of depth zero.

Theorem 2 (Block rule)**.** *If both boundary keys exist and their weights satisfy*

$$p_{i-1} \geq W + C, \qquad p_j \geq W + C,$$

there is an optimum in which all of B forms one subtree. For a prefix or suffix block, it suffices that the sole boundary key has weight at least C. Contracting the block to one gap of weight W then gives

$$\mathrm{OPT}(\textit{original}) = \mathrm{OPT}(\textit{contracted}) + C.$$

Proof. In any tree, let v be the lowest common ancestor of gaps q_i, q_j, at depth d. Its key lies in B, and every outcome of B descends from v. Hence the original contribution of B is at least $(d+1)W$.

Within the subtree rooted at v, delete the keys of B from the inward fringes of its two children. On the left, a deleted key has no retained right child and can be suppressed in favor of its left child; on the right the process is symmetric. Let the retained outside trees be A, D. All their retained outcomes are at depths no greater than before. If A is nonempty, its maximum is boundary key $i - 1$; if D is nonempty, its minimum is boundary key j.

Suppose both are nonempty, and their boundary keys had original depths $d + l, d + r$. If $l \geq r$, remove the maximum key from A, replacing its position by its left child, and make that key the new root at v. Its left child is the remainder of A and its right child is D. Put an optimal tree for B at the leftmost external position of D. This position has depth at most $d + r + 1$. No outside outcome other than the promoted boundary moves deeper, and that boundary saves lp_{i-1}. The increase in the contribution of B is at most

$$C + (d + r + 1)W - (d + 1)W = C + rW.$$

Since $l \geq r \geq 1$ and $p_{i-1} \geq W + C$, the saving is at least $l(W + C) \geq rW + C$. The mirrored construction applies for $r \geq l$.

If only one retained outside tree is nonempty, promote its inward boundary key to v and attach the optimal block as the other child. The block increase is at most C, while that boundary saves at least its own weight. The interior hypothesis suffices, and for a global prefix or suffix the weaker stated hypothesis suffices. If both outside trees are empty, B already occupies the subtree at v and can be optimized in place.

The construction never increases cost. Apply it to an optimum. A block at depth e contributes $C + eW$, while its replacement gap contributes eW. Contraction and expansion now prove the equality. □

A balanced k-key tree has height $h_k = \lceil \log_2(k+1) \rceil$ and cost at most $h_k W$. Thus the interior test

$$\min(p_{i-1}, p_j) \geq (1 + h_k)W$$

and endpoint test $p_{\text{boundary}} \geq h_k W$ certify a block without first solving it. These are sufficient conditions, not characterizations.

4 An adaptive linear selection pass

Build the maximum Cartesian tree of the key-weight sequence, breaking ties deterministically. Its subtree intervals form a laminar family: intervals are disjoint or nested. Compute their sizes and masses by one traversal and prefix sums. Test each proper interval with the balanced-tree bound above. This inspects only $O(n)$ candidates. The restriction to Cartesian intervals is an algorithmic choice, not a claim that all useful blocks occur there.

For a threshold b, select the maximal certified intervals whose sizes are at most b. Different selected intervals have a surviving separator key between them. Contracting one therefore leaves every other selected interval's weights and boundary-key weights unchanged, so the block theorem applies successively. Write their sizes as $k_1, \ldots, k_s$, and $m = n - \sum_t k_t$. The exact number of nonempty interval-DP states is

$$S(b) = \tau(m) + \sum_t \tau(k_t), \qquad \tau(u) = \frac{u(u+1)}{2}.$$

We choose a threshold minimizing $S(b)$ without running these DPs. For every certified interval I of size k, find its maximal proper certified descendants J. Immediately before I becomes eligible, these descendants are its selected blocks. Its activation therefore changes the core size and block-state total by

$$\Delta m = -\left(k - \sum_J |J|\right), \qquad \Delta S_{\text{blocks}} = \tau(k) - \sum_J \tau(|J|).$$

These sums are computed bottom-up. Bucket activation events by integer size $1, \ldots, n$ and scan the buckets, maintaining m and the block-state total. There are $O(n)$ events and buckets. Equal-size intervals are disjoint; evaluate a threshold after processing its entire bucket. The empty selection $b = 0$ is included.

Corollary 3. *The selected threshold and blocks are found in $O(n)$ arithmetic operations and space. Solving the blocks and surviving core by Knuth's method and expanding their saved trees takes*

$$O\left(n + m^2 + \sum_t k_t^2\right)$$

time and $O(n + m^2 + \max_t k_t^2)$ space when block tables are released after each solve. The chosen state count is no larger than $n(n+1)/2$. Worst-case time remains $O(n^2)$.

The minimization is only over these size thresholds in this candidate hierarchy, not over every possible partition. It does not minimize wall-clock time. Preprocessing, table allocation, root scans, and reconstruction still count as work.

For perspective, a certified partition with $m = O(n^{2/3})$ surviving keys and every $k_t = O(n^{1/3})$ gives $O(n^{4/3})$ time, since $\sum k_t^2 \leq (\max k_t)n$. This is a conditional structural consequence, not an unrestricted bound.

5 Recursive construction with a bounded preprocessing depth

The implementation `recursive_contract.py` combines the proved components. At each call it applies the single-key pass, compares the order-free lower bound of Section 7 with a feasible bisection-tree cost, and returns immediately on equality. Otherwise it selects blocks as above. It recurses only if their combined table-state estimate is at most three quarters of the full estimate; small components and unsuccessful decompositions fall back to Knuth's method.

Suppose a component has $m > 8$ keys after the single-key pass and is successfully decomposed. Every child component of size s satisfies $s(s+1) \le \frac{3}{4}m(m+1)$, hence $s < 0.92m$. Child sizes sum to m. Consequently the recursion has logarithmic depth, and the total number of keys processed on any recursion level is at most n. Linear selection and reconstruction cost $O(n \log n)$ in total. The heap-based equality checks cost $O(n \log^2 n)$ over all levels. If the terminal DP sizes are r_t, the construction therefore takes

$$O\left(n \log^2 n + \sum_t r_t^2\right)$$

arithmetic operations. Its worst case is still quadratic. This threshold prevents repeated expensive scans after arbitrarily small reductions. Correctness follows by contraction, exact equality certificates, and induction on the strictly smaller recursive components; reconstruction restores the original key indices and gap subtrees.

The recursive implementation passed 3,000 further random inputs through $n = 30$ against the cubic oracle and direct tree evaluation. A separate nested fixture checks composition across multiple decomposition levels. Matched Python timings use five inputs per family, alternate which solver runs first, and include preprocessing and reconstruction for both solvers. At $n = 1024$, the median runtime ratio (recursive divided by Knuth) was 0.134 on the positive-weight heavy-separator family; its median terminal DP count was 30,017 versus 524,800. On random mixed weights the ratio was 0.986 and on near-uniform weights 1.001, which support no meaningful gain. At $n = 512$, the respective random and near-uniform ratios were 1.131 and 1.093: preprocessing can make the method slower. These are exploratory measurements on one M4 host, not production-language or cross-machine claims.

6 Exact checks and preliminary measurements

The code uses nonnegative integer weights, an independent cubic recurrence, Knuth's recurrence, and direct evaluation of reconstructed trees. The basic harness additionally enumerates all tree shapes for small sizes. CPU experiments ran on the sponsor's M4 Mac mini. The reported quantities are algorithmic counts, not inferred asymptotic bounds from elapsed time.

- Single-key contraction: 22,143 exhaustive weight assignments in $\{0, 1, 2\}$ through $n = 4$, and 6,000 seeded random instances through $n = 20$. Every cost and reconstructed tree matched the cubic oracle.
- Block theorem: 19,486 qualifying interval contractions across 4,500 seeded instances through $n = 16$ matched independent cubic costs.
- Block algorithm: 3,000 random instances through $n = 20$, including fixed and adaptive thresholds, matched the cubic oracle and tree evaluation. Additional small cases check threshold selection against every threshold.

Table 1 records one seeded instance per family at $n = 512$. The baseline has 131,328 states. The block column includes all states inside selected blocks, not just the core. Results from different solvers use their separately recorded seeds and are not paired timing experiments.

Table 1: Preliminary exact DP-state counts, including block solves.

Family	Single-key pass + DP	Adaptive blocks + DP
Uniform mixed weights	131,328	131,328
Near-uniform mixed weights	131,328	131,328
Random mixed weights	128,778	130,817
Log-uniform weights	116,886	94,964
Keys only (control)	92,235	88,508
Sparse heavy separators	—	10,344

In the heavy-separator instance all gaps and ordinary keys have integer weights in $\{1, 2, 3\}$, and 22 randomly selected keys have weight 10^6. The block algorithm retains exactly 22 keys. Its state reduction is about 12.7-fold, but this is not a claim of a 12.7-fold runtime speedup. On ordinary random mixed weights the gain is small. The single-key pass contracts every key for the tested sequence $p_i = 4^i, q_i = 4^i$, but not for most of the ratio-two sequence. Geometric integer inputs also illustrate why bit complexity must be distinguished from arithmetic-operation complexity.

7 A useful relaxation that failed as a general solver

Allow arbitrary assignment of key weights to internal positions and gap weights to external positions. In this relaxed problem, repeatedly take the two smallest current external weights a, b and the smallest remaining internal weight p, replace them by $a + b + p$, and add that sum to the cost.

To prove the rule, take a deepest internal node of an optimum. Its children are leaves. Swapping the smallest internal weight into it and the two smallest external weights into its children cannot increase cost. Contract that cherry and induct. Sorting and a heap give $O(n \log n)$ time. This elementary extension of the Huffman exchange argument is not claimed as novel.

The relaxed cost is a valid lower bound for full OBST. Equality with a feasible ordered tree certifies optimality. It agreed with an independent shape-enumeration and rearrangement oracle on 22,143 exhaustive and 2,700 random instances in the basic harness.

However, using the relaxation on demand in an exact sparse DP was poor on random inputs. At $n = 128$ it visited roughly 4,300–4,600 exact intervals instead of Knuth's 8,256, but spent approximately 3.9–4.1 million weight comparisons constructing bounds and inspected over 100,000 candidate roots. Fewer exact states did not mean less computation.

Nor does the relaxation's interval matrix inherit the required quadrangle inequality. For $p = (9, 1, 12)$, $q = (5, 1, 18, 10)$, its costs H satisfy

$$H(1, 2) + H(2, 3) = 60 > 54 = H(1, 3) + H(2, 2).$$

Thus applying Knuth-style matrix optimization directly to this bound would be unjustified.

Two additional exact counterexamples exclude tempting local certificates. With $p = (3, 5, 18)$, $q = (13, 14, 4, 8)$, the tree with root 2 and left subtree $0(\varnothing, 1)$ costs 127, every single-rotation neighbor costs at least 134, yet the balanced tree costs 125. With $p = (18, 8, 18)$, $q = (17, 11, 1, 13)$, the balanced tree costs 164 and every distinct tree obtained by promoting a descendant to the root of an existing subtree costs at least 169; the tree $0(\varnothing, 2(1, \varnothing))$ costs 157. These are barriers to those local certificates, not complexity lower bounds.

8 An ordered depth-chain lower bound

Let l_i be the comparison length of key i and g_i the depth of gap i. For every valid BST, adjacent keys have different lengths and

$$g_0 = l_0, \quad g_n = l_{n-1}, \quad g_i = \max(l_{i-1}, l_i) \quad (0 < i < n).$$

This identity is existing work; see Becker [4], Lemma 1. Let K_d count keys of length d and G_d count gaps at depth d. The root and child-count identities are

$$K_1 = 1, \qquad G_d + K_{d+1} - 2K_d = 0 \quad (1 \le d \le n), \qquad K_{n+1} = 0.$$

Permit any sequence in $\{1, \dots, n\}^n$ with unequal adjacent entries. For arbitrary multipliers $\lambda_0, \dots, \lambda_n$, minimize

$$\sum_i p_i l_i + \sum_i q_i g_i + \lambda_0(K_1 - 1) + \sum_{d=1}^{n} \lambda_d(G_d + K_{d+1} - 2K_d). \tag{3}$$

Call the minimum $L(\lambda)$. Every actual tree is permitted and has zero penalty, so $L(\lambda) \le \mathrm{OPT}$ for every multiplier vector. The expression is a chain with key energy $U_i(b) = p_i b + \lambda_{b-1} - 2\lambda_b$, gap energy $V_i(b) = q_i b + \lambda_b$, and constant $-\lambda_0$. For an interior transition into length b, take the smaller of

$$\min_{a<b} F(a) + V_i(b), \qquad \min_{a>b}\{F(a) + V_i(a)\}.$$

Prefix and suffix minima give $O(H)$ work per key for a domain of H lengths. Using $H = n$ would cost quadratic time even for one bound evaluation.

Theorem 4 (Exact affine-tail compression)**.** *Fix $h \ge 1$, arbitrary $\lambda_0, \dots, \lambda_h$, and a slope $0 \le s \le \min_i p_i$. Extend $\lambda_d = \lambda_h + s(d - h)$ for $d > h$. The minimum in (3) is unchanged if lengths are restricted to $1, \dots, \min(n, h + 3)$.*

Proof. For $b \ge h + 1$, the unary key energy has slope $p_i - s \ge 0$ and the gap energy has slope $q_i + s \ge 0$. Both are nondecreasing in this tail. If some $l_i > h + 3$, replace it by a member of $\{h + 1, h + 2, h + 3\}$ unequal to its current neighbors. At most two values are forbidden. The key energy does not increase. Each adjacent maximum is either unchanged or decreases within the affine tail, so neither gap energy increases. Endpoint gaps obey the same argument. Repeat for all large lengths. The restricted domain is a subset of the original domain, and every original sequence has a no-more-expensive restricted sequence; therefore the minima are equal. □

Taking $h = \lceil \log_2(n+1) \rceil + 2$ gives an $O(n \log n)$ oracle. This is a statement about minimization of the relaxed energy, with no height assumption on the unknown optimal BST. Optimizing only the head multipliers requires folding all tail constraint violations into the derivative for λ_h. The implementation uses exact integer multipliers and costs scaled by 1024. Its subgradient ascent is heuristic; the bound is valid at every iteration regardless of convergence.

A stronger oracle requires exactly one length1 key. For each candidate root, the left and right chains then contain only lengths at least2. One forward and one reverse pass supply their closing costs for every root position, giving all forced-root bounds in $O(nH)$ total time. The projection above preserves the unique root, so tail compression remains valid for each marginal. Retaining the maximum bound seen for each root, then taking the minimum over roots, is also a valid lower bound.

Both oracles passed 50 comparisons against exhaustive length-sequence minimization, 5,000 arbitrary-multiplier bound checks against cubic OBST, and 1,050 direct comparisons of the full and shortened oracles. Forced-root marginals were additionally checked against exact root-constrained costs.

A structural gap. The sequence $l = (3, 1, 2, 3, 2)$ has $g = (3, 3, 2, 3, 3, 2)$, a unique root, and all the level-count equalities. It is not a valid BST depth sequence. For

$$p = (3, 27, 21, 16, 12), \qquad q = (18, 22, 27, 22, 12, 22),$$

it costs 470, while the true optimum is 472. Its penalty is zero for every multiplier vector. Hence neither oracle family can ever exceed 470 on this instance. More optimization iterations cannot remove this obstruction.

Preliminary strength and cost. At $n = 256$, normalized gaps $(\mathrm{OPT} - \mathrm{LB})/W$ for the chain bound versus order-free Huffman were 0.15003 versus 0.22080 on a random mixed instance, and 0.00873 versus 0.49294 on an alternating instance. However, 300 ascent iterations used 1,082,368 key-depth cells including initialization, compared with 32,896 interval states for Knuth. The unique-root variant uses two passes and still leaves 254 of 256 potentially improving roots on the random instance, and 255 on the alternating instance. Only two survive on the gap-spike instance. These results identify a lower-bound component and its limitations; they do not establish a faster general exact solver.

9 Status and reproducibility

This investigation has not proved that subquadratic full OBST construction is impossible, nor supplied a general subquadratic construction. The positive contribution of this checkpoint is an exact decomposition argument, an adaptive implementation, and reproducible evidence of its scope and limitations. Mathematical novelty, broad practical benefit, and comparison against Larmore-style implementations remain to be assessed.

All programs use only the Python standard library. The code and seeded results are stored under `obst/code` and `obst/results`. The repository README provides commands for independent oracles, exact construction, matched timing, and both depth-price models. The active objective remains to avoid most of the quadratic table on a substantially broader class, or establish a rigorously scoped lower bound.

References

[1] D. E. Knuth. Optimum binary search trees. *Acta Informatica* 1, 14–25, 1971. doi:10.1007/BF00264289.

[2] L. L. Larmore. A subquadratic algorithm for constructing approximately optimal binary search trees. *Journal of Algorithms* 8(4), 579–591, 1987. doi:10.1016/0196-6774(87)90052-6.

[3] M. Karpinski, L. L. Larmore, and W. Rytter. Sequential and parallel subquadratic work algorithms for constructing approximately optimal binary search trees. *Proceedings of SODA*, 36–41, 1996. Author-hosted manuscript.

[4] P. Becker. Optimal binary search trees with near minimal height. 2010. arXiv:1011.1337.